

Базовая лабораторная работа 2

Выполнил: Батаргин Егор Александрович
ИСУ:335189
Группа: P3330

Задание

Реализация блочного кэша в пространстве пользователя в виде динамической библиотеки. Операции чтения и записи на диск. Такой кэш позволяет избежать высоких задержек при повторном доступе к данным, так как операция будет выполнена с данными в RAM, а не на диске.

В данной лабораторной работе необходимо реализовать блочный кэш в пространстве пользователя в виде динамической библиотеки. Политику вытеснения страниц и другие элементы задания необходимо получить у преподавателя.

При выполнении работы необходимо реализовать простой API для работы с файлами, предоставляющий пользователю следующие возможности (по аналогии с системным API):

- Открытие файла по заданному пути файла, доступного для чтения. Процедура возвращает некоторый хэндл на файл. Пример: `int vtpc_open(const char *path)`.
- Закрытие файла по хэндлу. Пример: `int vtpc_close(int fd)`.
- Чтение данных из файла. Пример: `ssize_t vtpc_read(int fd, void buf[.count], size_t count)`.
- Запись данных в файл. Пример: `ssize_t vtpc_write(int fd, const void buf[.count], size_t count)`.
- Перестановка позиции указателя на данные файла. Достаточно поддерживать только абсолютные координаты. Пример: `off_t vtpc_lseek(int fd, off_t offset, int whence)`.
- Синхронизация данных из кэша с диском. Пример: `int vtpc_fsync(int fd)`.
- Операции с диском разработанного блочного кэша должны производиться в обход страничного кэша операционной системы.

В рамках проверки работоспособности разработанного блочного кэша необходимо адаптировать указанную преподавателем программу-загрузчик из ЛР 1, добавив использование кэша. Запустите программу и убедитесь, что она корректно работает. Сравните производительность до и после.

Точкой входа в ваш модуль являются файлы `vtpc.h` и `vtpc.c`. Именно этот API вызывается тестовым фреймворком, сравнивающим поведения вашего `vtpc` и `libc`.

Ограничения

Программа (комплекс программ) должна быть реализован на языке C.

Если по выданному варианту задана политика вытеснения Optimal, то необходимо предоставить пользователю возможность подсказать page cache, когда будет совершен следующий доступ к данным. Это можно сделать либо добавив параметр в процедуры read и write (например, ssize_t vtpc_read(int fd, void buf[.count], size_t count, access_hint_t hint)), либо добавив еще одну функцию в API (например, int vtpc_advice(int fd, off_t offset, access_hint_t hint)). access_hint_t в данном случае – это абсолютное время или временной интервал, по которому разработанное API будет определять время последующего доступа к данным.

Запрещено использовать высокоуровневые абстракции над системными вызовами.

Выполнение работы

Основным файлом для работы считается vtpc.c, в котором реализуется API для работы с файлами.

Среди библиотек данного файла можно найти stdio.h - скажу сразу, использовалась для отладочного вывода.

vtpc.c

По сути мы сделали свою абстракцию для системных вызовов.

Начнем по порядку

```
// Размер блока (обычно соответствует размеру страницы или сектору диска)
#define BLOCK_SIZE 4096

// Максимальное количество блоков в кэше
#define MAX_CACHE_BLOCKS 1024
```

Размер блока 4096 выбран не случайно, так как он соответствует размеру страницы в операционных системах.

Размер взят произвольный. Для кеша составит $1024 * 4 = 4$ МБ кеша.

Дальше мы создаем структуру для хранения информации о блоке кеша.

```
typedef struct CacheBlock {
    int fd; // Файловый дескриптор
    off_t block_number; // Номер блока в файле
    char data[BLOCK_SIZE]; // Данные блока
    int dirty; // Флаг "грязного" блока (изменён, но не записан)
    time_t last_access; // Время последнего доступа (для LRU)
    struct CacheBlock *prev; ///< Указатель на предыдущий блок в списке LRU
    struct CacheBlock *next; ///< Указатель на следующий блок в списке LRU
} CacheBlock;
```

Как видно в комментариях, мы храним то, в каком файловом дескрипторе находится блок, номер блока в файле, данные блока, флаг "грязного" блока (изменен, но не записан), время последнего доступа (для LRU), указатели на предыдущий и следующий блоки в списке LRU.

Что такое дескриптор? Это идентификатор файла, по которому потом можно обратиться. Указывает на то, какой файл у нас в этом блоке.

Дальше мы создаем структуру для хранения информации о файле

```
typedef struct FileInfo {
    int fd; // Системный файловый дескриптор
    off_t file_size; // Размер файла
    off_t position; // Текущая позиция в файле
} FileInfo;
```

Я думаю, по комментариям все понятно.

```
typedef struct CacheManager {
    CacheBlock *head; // Начало списка LRU (самый недавно использованный)
    CacheBlock *tail; // Конец списка LRU (самый давно использованный)
    int block_count; // Текущее количество блоков в кэше
    volatile int lock; // Спинлок для потокобезопасности
} CacheManager;
```

Это структура для управления кешем. Тут мы будем содержать и обновлять информацию о том, какой файл был недавно использованным, а какой - давно. Это нам необходимо для реализации LRU. LRU - это алгоритм вытеснения данных. При заполненном кеше мы вытесняем из него самые давно использованные данные. Для начала нам надо установить начальные значения для всех указателей и счетчиков.

```
static void init_cache(void) {
    cache_manager.head = NULL;
    cache_manager.tail = NULL;
    cache_manager.block_count = 0;
    spinlock_init(&cache_manager.lock);

    // Инициализируем массив открытых файлов
    for (int i = 0; i < 1024; i++) {
        open_files[i] = NULL;
    }
}
```

Дальше реализуем наш очиститель кеша.

```
void vtpc_cleanup(void) {
    spin_lock(&cache_manager.lock);

    // Записываем все "грязные" блоки на диск
    CacheBlock *current = cache_manager.head;
    while (current != NULL) {
        if (current->dirty) {
            write_block_to_disk(current);
        }
        CacheBlock *next = current->next;
        raw_munmap(current, sizeof(CacheBlock));
        current = next;
    }

    cache_manager.head = NULL;
    cache_manager.tail = NULL;
    cache_manager.block_count = 0;

    spin_unlock(&cache_manager.lock);

    // Закрываем все открытые файлы
    for (int i = 0; i < 1024; i++) {
        if (open_files[i] != NULL) {
            FileInfo *info = open_files[i];
            raw_close(info->fd);
            raw_munmap(info, sizeof(FileInfo));
            open_files[i] = NULL;
        }
    }
}
```

Эта функция выполняет полную очистку кэша, записывая все "грязные" блоки на диск и освобождая память.

Процесс очистки кэша:

1. Блокируем кэш для потокобезопасности:

```
spin_lock(&cache_manager.lock);
```

2. Записываем все "грязные" блоки на диск и освобождаем память:

```
CacheBlock *current = cache_manager.head;
while (current != NULL) {
    if (current->dirty) {
        write_block_to_disk(current);
    }
    CacheBlock *next = current->next;
    raw_munmap(current, sizeof(CacheBlock));
    current = next;
}
```

3. Сбрасываем параметры кэша:

```
cache_manager.head = NULL;
cache_manager.tail = NULL;
cache_manager.block_count = 0;
```

4. Разблокируем кэш:

```
spin_unlock(&cache_manager.lock);
```

5. Закрываем все открытые файлы и освобождаем память:

```
for (int i = 0; i < 1024; i++) {
    if (open_files[i] != NULL) {
        FileInfo *info = open_files[i];
        raw_close(info->fd);
        raw_munmap(info, sizeof(FileInfo));
        open_files[i] = NULL;
    }
}
```

Теперь кратко об функциях блокировки кеша и разблокировки кеша.

Дабы избежать использования высокоуровневых абстракций над системными вызовами, мы сделаем свою абстракцию над ними.

```
static inline void spin_lock(volatile int *lock) {
    while (__sync_lock_test_and_set(lock, 1)) {
        #ifdef __x86_64__
            __asm__ __volatile__ ("pause" ::: "memory");
        #endif
    }
}

static inline void spin_unlock(volatile int *lock) {
    __sync_lock_release(lock);
}
```

В цикле `while` в функции `spin_lock` мы проверяем, заблокирован ли кеш. Если заблокирован, то мы выполняем паузу и проверяем снова. Если кеш не заблокирован, то мы заблокируем его и выходим из цикла. Такой механизм защищает от гонки данных. Важно отметить, что `__sync_lock_test_and_set` - атомарная операция, которая обрабатывается напрямую компилятором. Это вложено в GCC и не является высокоуровневой абстракцией.

Поехали дальше

Реализуем функцию для поиска блока в кеше

```
static CacheBlock* find_block_in_cache(int fd, off_t block_number) {
    CacheBlock *current = cache_manager.head;

    while (current != NULL) {
        if (current->fd == fd && current->block_number == block_number) {
            // Обновляем время доступа
            current->last_access = time(NULL);
            move_to_front(current);
            return current;
        }
        current = current->next;
    }

    return NULL;
}
```

На вход программа принимает файловый дескриптор и номер блока. Функция ищет блок в кеше и если он есть, то обновляет время доступа и перемещает блок в начало списка. Если блок не найден, то возвращает NULL.

Про функцию `move_to_front` как раз сейчас поговорим.

```

static void move_to_front(CacheBlock *block) {
    if (block == cache_manager.head) {
        return; // Уже в начале
    }

    // Удаляем блок из текущей позиции
    if (block->prev) {
        block->prev->next = block->next;
    }
    if (block->next) {
        block->next->prev = block->prev;
    }

    // Если блок был хвостом, обновляем хвост
    if (block == cache_manager.tail) {
        cache_manager.tail = block->prev;
    }

    // Вставляем блок в начало
    block->prev = NULL;
    block->next = cache_manager.head;

    if (cache_manager.head) {
        cache_manager.head->prev = block;
    }

    cache_manager.head = block;

    // Если список был пуст, блок становится и хвостом
    if (cache_manager.tail == NULL) {
        cache_manager.tail = block;
    }
}

```

А теперь по тому, как оно работает

Сначала выполним проверку, что блок не в начале списка.

Проверим, есть ли у блока предыдущий блок.

Если есть, то ставим у предыдущего блока следующим тот блок, что следующий от текущего. И то же самое касается предыдущего.

ДЛЯ СЕБЯ

`block->prev->next = block->next;` - игра указателей.

Представим, что есть блок А, В и С

```
A -> B -> C
```

У блока В prev указывает на блок А, а next на блок С.

block->prev - указатель на предыдущий блок A

block->prev->next - указатель на следующий блок относительно блока A. В который мы сохраняем block->next = C (относительно текущего блока B)

```
block->prev - A
```

```
block->next - C
```

```
block->prev->next = block->next
```

Теперь у блока A next указывает на блок C, а блока C prev указывает на блок A

Дальше идем

Если блок был хвостом, то новый хвост - это будет предыдущий блок относительно изымаемого (текущего)

Обновляем данные блока:

- удаляем у блока предыдущий (он же голова)
- Теперь его следующий блок - это прошлый головной блок.
- У того блока, что еще в голове делаем предыдущим блок тот, что вставляем в начало
- обновления head на новый блок

Ну и если список был пуст, блок становится и хвостом еще

Сделаем простую функцию, что будет вычислять номер блока по его позиции:

```
static off_t get_block_number(off_t position) {  
    return position / BLOCK_SIZE;  
}
```

Тут все просто.

А смещение внутри блока - это остаток от деления позиции на размер блока.

```
static off_t get_block_offset(off_t position) {  
    return position % BLOCK_SIZE;  
}
```

Реализуем функцию чтения блока из диска физического:


```

static void read_block_from_disk(CacheBlock *block) {
    FileInfo *file_info = open_files[block->fd];
    if (!file_info) return;

    off_t pos = block->block_number * BLOCK_SIZE;
    off_t old_pos = raw_lseek(file_info->fd, 0, SEEK_CUR);

    if (old_pos < 0 || raw_lseek(file_info->fd, pos, SEEK_SET) < 0) {
        if (old_pos >= 0) raw_lseek(file_info->fd, old_pos, SEEK_SET);
        raw_memset(block->data, 0, BLOCK_SIZE);
        return;
    }

    ssize_t bytes = raw_read(file_info->fd, block->data, BLOCK_SIZE);
    raw_lseek(file_info->fd, old_pos, SEEK_SET);

    if (bytes < BLOCK_SIZE) {
        if (bytes > 0) {
            raw_memset(block->data + bytes, 0, BLOCK_SIZE - bytes);
        } else {
            raw_memset(block->data, 0, BLOCK_SIZE);
        }
    }

    block->dirty = 0;
    block->last_access = raw_time();
}

```

Про чтение

На вход принимает блок.

```

FileInfo *file_info = open_files[block->fd];
if (!file_info) return;

```

Дальше вычисление позиции на диске

```

off_t pos = block->block_number * BLOCK_SIZE;

```

Сохраняем текущую позицию в файле, чтобы потом вернуться

```

off_t old_pos = raw_lseek(file_info->fd, 0, SEEK_CUR);

```

Перемещаемся в нужную позицию для чтения блока

```

if (old_pos < 0 || raw_lseek(file_info->fd, pos, SEEK_SET) < 0) {
    if (old_pos >= 0) raw_lseek(file_info->fd, old_pos, SEEK_SET);
    raw_memset(block->data, 0, BLOCK_SIZE);
    return;
}

```

Читаем данные блока

```
ssize_t bytes = raw_read(file_info->fd, block->data, BLOCK_SIZE);
```

Возвращаемся в исходную позицию

```
raw_lseek(file_info->fd, old_pos, SEEK_SET);
```

Если прочитали меньше, чем размер блока, заполняем остаток нулями

```
if (bytes < BLOCK_SIZE) {  
    if (bytes > 0) {  
        raw_memset(block->data + bytes, 0, BLOCK_SIZE - bytes);  
    } else {  
        raw_memset(block->data, 0, BLOCK_SIZE);  
    }  
}
```

Сбрасываем флаг "грязного" блока и обновляем время последнего доступа

```
block->dirty = 0;  
block->last_access = raw_time();
```

Реализуем функцию записи блока на диск:

```
static void write_block_to_disk(CacheBlock *block) {  
    FileInfo *file_info = open_files[block->fd];  
    if (!file_info || !block->dirty) return;  
  
    off_t pos = block->block_number * BLOCK_SIZE;  
    off_t old_pos = raw_lseek(file_info->fd, 0, SEEK_CUR);  
  
    if (old_pos < 0 || raw_lseek(file_info->fd, pos, SEEK_SET) < 0) {  
        if (old_pos >= 0) raw_lseek(file_info->fd, old_pos, SEEK_SET);  
        return;  
    }  
  
    ssize_t written = raw_write(file_info->fd, block->data, BLOCK_SIZE);  
    raw_lseek(file_info->fd, old_pos, SEEK_SET);  
  
    if (written == BLOCK_SIZE) {  
        block->dirty = 0;  
    }  
}
```

Про запись

На вход принимает блок.

```
FileInfo *file_info = open_files[block->fd];  
if (!file_info || !block->dirty) return;
```

Проверяем, что файл открыт и блок "грязный" (требуется запись на диск).

Вычисляем позицию на диске и сохраняем текущую позицию в файле:

```
off_t pos = block->block_number * BLOCK_SIZE;
off_t old_pos = raw_lseek(file_info->fd, 0, SEEK_CUR);
```

Перемещаемся в нужную позицию для записи блока:

```
if (old_pos < 0 || raw_lseek(file_info->fd, pos, SEEK_SET) < 0) {
    if (old_pos >= 0) raw_lseek(file_info->fd, old_pos, SEEK_SET);
    return;
}
```

Записываем данные блока на диск:

```
ssize_t written = raw_write(file_info->fd, block->data, BLOCK_SIZE);
```

Возвращаемся в исходную позицию:

```
raw_lseek(file_info->fd, old_pos, SEEK_SET);
```

Если запись прошла успешно, сбрасываем флаг "грязного" блока:

```
if (written == BLOCK_SIZE) {
    block->dirty = 0;
}
```

Реализуем функцию вытеснения блока из кэша:

```
static void evict_lru_block(void) {
    if (cache_manager.tail == NULL) {
        return;
    }

    CacheBlock *to_evict = cache_manager.tail;

    // Если блок "грязный", записываем его на диск
    if (to_evict->dirty) {
        write_block_to_disk(to_evict);
    }

    // Удаляем блок из списка
    if (to_evict->prev) {
        to_evict->prev->next = NULL;
    }

    cache_manager.tail = to_evict->prev;

    if (cache_manager.head == to_evict) {
        cache_manager.head = NULL;
    }

    // Освобождаем память
    raw_munmap(to_evict, sizeof(CacheBlock));
    cache_manager.block_count--;
}
```

Про вытеснение

Эта функция реализует алгоритм LRU (Least Recently Used) для вытеснения блоков из кэша.

Проверяем, что в кэше есть блоки:

```
if (cache_manager.tail == NULL) {  
    return;  
}
```

Выбираем блок для вытеснения (самый давно использованный - в хвосте списка):

```
CacheBlock *to_evict = cache_manager.tail;
```

Если блок "грязный", записываем его на диск перед вытеснением:

```
if (to_evict->dirty) {  
    write_block_to_disk(to_evict);  
}
```

Удаляем блок из списка LRU:

```
if (to_evict->prev) {  
    to_evict->prev->next = NULL;  
}  
  
cache_manager.tail = to_evict->prev;  
  
if (cache_manager.head == to_evict) {  
    cache_manager.head = NULL;  
}
```

Освобождаем память, занятую блоком:

```
raw_munmap(to_evict, sizeof(CacheBlock));  
cache_manager.block_count--;
```

Реализуем функцию выделения нового блока:

```

static CacheBlock* allocate_new_block(int fd, off_t block_number) {
    spin_lock(&cache_manager.lock);

    // Если кэш полон, вытесняем LRU блок
    if (cache_manager.block_count >= MAX_CACHE_BLOCKS) {
        evict_lru_block();
    }

    // Создаем новый блок
    CacheBlock *new_block = raw_mmap(sizeof(CacheBlock));
    if (!new_block) {
        spin_unlock(&cache_manager.lock);
        return NULL;
    }

    // Инициализируем блок
    raw_memset(new_block, 0, sizeof(CacheBlock));
    new_block->fd = fd;
    new_block->block_number = block_number;
    new_block->last_access = raw_time();

    // Читаем данные с диска
    read_block_from_disk(new_block);

    // Добавляем блок в начало списка
    new_block->next = cache_manager.head;
    if (cache_manager.head) {
        cache_manager.head->prev = new_block;
    }

    cache_manager.head = new_block;

    if (cache_manager.tail == NULL) {
        cache_manager.tail = new_block;
    }

    cache_manager.block_count++;

    spin_unlock(&cache_manager.lock);
    return new_block;
}

```

Про выделение

Эта функция выделяет новый блок в кэше, при необходимости вытесняя старые блоки.

Блокируем кэш для потокобезопасности:

```
spin_lock(&cache_manager.lock);
```

Проверяем, не превышен ли лимит блоков в кэше:

```
if (cache_manager.block_count >= MAX_CACHE_BLOCKS) {  
    evict_lru_block();  
}
```

Создаем новый блок в памяти:

```
CacheBlock *new_block = raw_mmap(sizeof(CacheBlock));  
if (!new_block) {  
    spin_unlock(&cache_manager.lock);  
    return NULL;  
}
```

Инициализируем поля блока:

```
raw_memset(new_block, 0, sizeof(CacheBlock));  
new_block->fd = fd;  
new_block->block_number = block_number;  
new_block->last_access = raw_time();
```

Читаем данные блока с диска:

```
read_block_from_disk(new_block);
```

Добавляем блок в начало списка LRU:

```
new_block->next = cache_manager.head;  
if (cache_manager.head) {  
    cache_manager.head->prev = new_block;  
}  
  
cache_manager.head = new_block;  
  
if (cache_manager.tail == NULL) {  
    cache_manager.tail = new_block;  
}
```

Увеличиваем счетчик блоков и разблокируем кэш:

```
cache_manager.block_count++;  
  
spin_unlock(&cache_manager.lock);  
return new_block;
```

API функции

`int vtpc_open(const char *path)` - открывает файл и возвращает дескриптор:

Эта функция открывает файл и возвращает виртуальный дескриптор файла для использования с другими функциями API.

Процесс открытия файла:

1. Проверяем, была ли инициализирована система кэширования:

```
static int initialized = 0;
if (!initialized) {
    raw_memset(&cache_manager, 0, sizeof(cache_manager));
    raw_memset(open_files, 0, sizeof(open_files));
    initialized = 1;
}
```

2. Пытаемся открыть файл с правами чтения/записи, если не удалось - только для чтения:

```
int sys_fd = raw_open(path, 2); // O_RDWR
if (sys_fd < 0) {
    sys_fd = raw_open(path, 0); // O_RDONLY
    if (sys_fd < 0) return -1;
}
```

3. Получаем размер файла:

```
off_t size = raw_lseek(sys_fd, 0, SEEK_END);
if (size < 0 || raw_lseek(sys_fd, 0, SEEK_SET) < 0) {
    raw_close(sys_fd);
    return -1;
}
```

4. Ищем свободный слот для FileInfo:

```
int vtpc_fd = -1;
for (int i = 0; i < 1024; i++) {
    if (!open_files[i]) {
        vtpc_fd = i;
        break;
    }
}
```

5. Выделяем память для FileInfo и инициализируем его:

```
FileInfo *info = raw_mmap(sizeof(FileInfo));
if (!info) {
    raw_close(sys_fd);
    return -1;
}

info->fd = sys_fd;
info->file_size = size;
info->position = 0;

open_files[vtpc_fd] = info;
return vtpc_fd;
```

`int vtpc_close(int fd)` - закрывает файл по дескриптору:

Эта функция закрывает файл и освобождает все связанные с ним ресурсы.

Процесс закрытия файла:

1. Проверяем корректность дескриптора:

```
if (fd < 0 || fd >= 1024 || !open_files[fd]) return -1;
```

2. Блокируем кэш **для** потокобезопасности:

```
spin_lock(&cache_manager.lock);
```

3. Записываем **все** "грязные" блоки этого файла на диск:

```
CacheBlock *curr = cache_manager.head;
while (curr) {
    if (curr->fd == fd && curr->dirty) {
        write_block_to_disk(curr);
    }
    curr = curr->next;
}
```

4. Удаляем блоки этого файла **из** кэша:

```
curr = cache_manager.head;
while (curr) {
    CacheBlock *next = curr->next;
    if (curr->fd == fd) {
        // Удаляем из списка
        if (curr->prev) curr->prev->next = curr->next;
        if (curr->next) curr->next->prev = curr->prev;
        if (curr == cache_manager.head) cache_manager.head = curr->next;
        if (curr == cache_manager.tail) cache_manager.tail = curr->prev;
```

```
        raw_munmap(curr, sizeof(CacheBlock));
        cache_manager.block_count--;
    }
    curr = next;
}
```

5. Разблокируем кэш:

```
spin_unlock(&cache_manager.lock);
```

6. Закрываем **файл** и освобождаем память:

```
FileInfo *info = open_files[fd];
raw_close(info->fd);
raw_munmap(info, sizeof(FileInfo));
open_files[fd] = 0;
```



```
`ssize_t vtpc_read(int fd, void *buf, size_t count)` - читает данные из файла:
```

Эта функция читает указанное количество байт из файла в буфер.

Процесс чтения данных:

1. Проверяем корректность параметров:

```
if (fd < 0 || fd >= 1024 || !open_files[fd] || !buf) return -1;
```

2. Определяем, сколько байт нужно прочитать:

```
FileInfo *info = open_files[fd];
if (info->position >= info->file_size) return 0;

size_t to_read = count;
if (info->position + (off_t)to_read > info->file_size) {
    to_read = info->file_size - info->position;
}
```

3. Читаем данные блоками:

```
size_t total = 0;
char buffer = (char)buf;

while (total < to_read) {
    off_t pos = info->position + (off_t)total;
    off_t block_num = get_block_number(pos);
    off_t offset = get_block_offset(pos);

    size_t in_block = BLOCK_SIZE - offset;
    size_t needed = to_read - total;
    size_t copy = (in_block < needed) ? in_block : needed;

    // Ищем блок в кэше
    CacheBlock *block = find_block_in_cache(fd, block_num);

    if (!block) {
        block = allocate_new_block(fd, block_num);
        if (!block) {
            info->position += (off_t)total;
            return total;
        }
    }

    raw_memcpy(buffer + total, block->data + offset, copy);
    total += copy;
}
```

4. Обновляем позицию в файле:

```
info->position += (off_t)total;  
return total;
```

`ssize_t vtpc_write(int fd, const void *buf, size_t count)` - записывает данные в файл:

Эта функция записывает указанное количество байт из буфера в файл.

Процесс записи данных:

1. Проверяем корректность параметров:

```
if (fd < 0 || fd >= 1024 || !open_files[fd] || !buf) return -1;
```

2. Записываем данные блоками:

```

FileInfo *info = open_files[fd];
size_t total_bytes_written = 0;
const char *buffer = (const char*)buf;

while (total_bytes_written < count) {
    off_t current_pos = info->position + (off_t)total_bytes_written;
    off_t block_num = get_block_number(current_pos);
    off_t block_offset = get_block_offset(current_pos);

    size_t bytes_in_block = BLOCK_SIZE - block_offset;
    size_t bytes_needed = count - total_bytes_written;
    size_t bytes_to_copy = (bytes_in_block < bytes_needed) ?
bytes_in_block : bytes_needed;

    // Ищем блок в кэше
    CacheBlock *block = find_block_in_cache(fd, block_num);

    // Если блока нет в кэше, создаем новый
    if (!block) {
        block = allocate_new_block(fd, block_num);
        if (!block) {
            info->position += (off_t)total_bytes_written;
            return total_bytes_written;
        }
    }

    // Копируем данные из буфера в блок
    raw_memcpy(block->data + block_offset,
                buffer + total_bytes_written,
                bytes_to_copy);

    // Помечаем блок как "грязный"
    block->dirty = 1;
    block->last_access = raw_time();

    // Перемещаем блок в начало списка LRU
    // ... (код перемещения блока)

    total_bytes_written += bytes_to_copy;

    // Обновляем размер файла, если нужно
    off_t new_end_pos = current_pos + (off_t)bytes_to_copy;
    if (new_end_pos > info->file_size) {
        info->file_size = new_end_pos;
    }
}

```

3. Обновляем позицию в файле:

```

info->position += (off_t)total_bytes_written;
return total_bytes_written;

```

off_t vtpc_lseek(int fd, off_t offset, int whence) - перемещает указатель позиции в файле:

Эта функция устанавливает позицию указателя в файле.

Поддерживаемые значения whence:

- SEEK_SET: позиция устанавливается относительно начала файла
- SEEK_CUR: позиция устанавливается относительно текущей позиции
- SEEK_END: позиция устанавливается относительно конца файла

Процесс перемещения указателя:

1. Проверяем корректность дескриптора:

```
if (fd < 0 || fd >= 1024 || !open_files[fd]) return -1;
```

2. Вычисляем новую позицию в зависимости от whence:

```
FileInfo *info = open_files[fd];
off_t new_position;

switch (whence) {
    case SEEK_SET:
        new_position = offset;
        break;

    case SEEK_CUR:
        new_position = info->position + offset;
        break;

    case SEEK_END:
        new_position = info->file_size + offset;
        break;

    default:
        return -1;
}
```

3. Проверяем, что позиция не отрицательна:

```
if (new_position < 0) {
    return -1;
}
```

4. Устанавливаем новую позицию:

```
info->position = new_position;
return new_position;
```

int vtpc_fsync(int fd) - синхронизирует данные файла с диском:

Эта функция записывает все "грязные" блоки файла на диск и вызывает системный fsync.

Процесс синхронизации:

1. Проверяем корректность дескриптора:

```
if (fd < 0 || fd >= 1024 || !open_files[fd]) return -1;
```

2. Блокируем кэш:

```
spin_lock(&cache_manager.lock);
```

3. Записываем все "грязные" блоки этого файла на диск:

```
CacheBlock *current = cache_manager.head;
while (current != NULL) {
    if (current->fd == fd && current->dirty) {
        write_block_to_disk(current);
    }
    current = current->next;
}
```

4. Разблокируем кэш:

```
spin_unlock(&cache_manager.lock);
```

5. Вызываем системный fsync:

```
FileInfo *info = open_files[fd];
int result = raw_fsync(info->fd);

return result;
```

Новые функции в библиотеке VTPC

Функции для работы с прямым доступом к диску

vtpc_open с флагами

Функция `int vtpc_open(const char *path, int flags)` открывает файл с дополнительными флагами:

- `VTPC_O_RDONLY` - открытие только для чтения
- `VTPC_O_WRONLY` - открытие только для записи
- `VTPC_O_RDWR` - открытие для чтения и записи
- `VTPC_O_DIRECT` - прямой доступ к диску (обход кэша ОС)
- `VTPC_O_SYNC` - синхронная запись

Прямой доступ к диску позволяет обходить кэш операционной системы, что может быть полезно для специфических задач, где требуется гарантированная запись данных на диск без использования промежуточного кэширования.

vtpc_read_aligned и vtpc_write_aligned

Функции `ssize_t vtpc_read_aligned(int fd, void *aligned_buf, size_t count)` и `ssize_t vtpc_write_aligned(int fd, const void *aligned_buf, size_t count)` работают с выровненными буферами памяти для прямого доступа к диску.

Выровненные буферы - это буферы памяти, адрес которых кратен определенному значению (в нашем случае 512 байт). Это требование для работы с прямым доступом к диску.

vtpc_read_direct и vtpc_write_direct

Функции `ssize_t vtpc_read_direct(int fd, void *aligned_buf, size_t count)` и `ssize_t vtpc_write_direct(int fd, const void *aligned_buf, size_t count)` обеспечивают принудительное чтение/запись напрямую с диска, обходя кэш, даже если файл открыт без флага `O_DIRECT`.

vtpc_alloc_aligned_buffer и vtpc_free_aligned_buffer

Функции `void* vtpc_alloc_aligned_buffer(size_t size)` и `void vtpc_free_aligned_buffer(void *buffer)` позволяют выделять и освобождать выровненную память для работы с прямым доступом к диску.

Как работает прямой доступ к диску

Прямой доступ к диску (`O_DIRECT`) позволяет обходить кэш операционной системы при чтении и записи данных. Это может быть полезно в следующих случаях:

1. **Гарантированная запись данных** - данные сразу записываются на диск, минуя кэш операционной системы.
2. **Специфические требования к производительности** - в некоторых случаях прямой доступ может быть быстрее, особенно при больших объемах данных.
3. **Тестирование производительности диска** - позволяет измерить реальную производительность диска без влияния кэша.

Однако, прямой доступ имеет и недостатки:

1. **Требования к выравниванию** - буферы памяти и размеры операций должны быть выровнены по определенным границам.
2. **Потенциально меньшая производительность** - кэш операционной системы часто улучшает производительность за счет уменьшения количества операций ввода-вывода.

Функции мониторинга производительности

vtpc_get_cache_size и vtpc_get_dirty_count

Функции `size_t vtpc_get_cache_size(void)` и `size_t vtpc_get_dirty_count(void)` позволяют получить информацию о текущем состоянии кэша:

- `vtpc_get_cache_size()` возвращает количество блоков в кэше
- `vtpc_get_dirty_count()` возвращает количество "грязных" блоков (требующих записи на диск)

vtpc_print_cache_stats

Функция `void vtpc_print_cache_stats(void)` выводит статистику кэша на экран.

Функция очистки ресурсов

`vtpc_cleanup`

Функция `void vtpc_cleanup(void)` записывает все "грязные" блоки на диск и освобождает все ресурсы, связанные с файловой системой. Эта функция должна вызываться при завершении работы с библиотекой.

Сборка проекта

Для сборки проекта используется Makefile со следующими целями:

- `make all` - сборка всех компонентов
- `make libvtpc.so` - сборка динамической библиотеки VTPC
- `make memory_loader_vtpc` - сборка тестовой программы MemoryLoader
- `make test` - запуск тестов
- `make clean` - очистка проекта

Использование библиотеки

Для использования библиотеки в своих программах необходимо:

1. Подключить заголовочный файл `vtpc.h`
2. Скомпилировать программу с использованием библиотеки `libvtpc.so`
3. При запуске программы убедиться, что библиотека доступна в системе

Пример использования:

```

#include "vtpc.h"

int main() {
    // Открытие файла с прямым доступом
    int fd = vtpc_open("test.txt", VTPC_O_RDWR | VTPC_O_DIRECT);

    // Выделение выровненного буфера
    char* buffer = vtpc_alloc_aligned_buffer(4096);

    // Чтение данных
    vtpc_read_direct(fd, buffer, 4096);

    // Запись данных
    vtpc_write_direct(fd, buffer, 4096);

    // Закрытие файла
    vtpc_close(fd);

    // Освобождение буфера
    vtpc_free_aligned_buffer(buffer);

    // Очистка ресурсов
    vtpc_cleanup();

    return 0;
}

```

Анализ

Для анализа я взял программу из БЛР1. Сейчас это MemoryLoader_vtpc.c. В ней реализованы две функции запуска:

- randomReadTest_vtpc - это запуск нагрузки с использованием нашего кеша
- randomReadTest_system - это запуск с системными вызовами, но в обход кеша.

Мы должны сравнить время работы.

Гипотеза

Вызов через системные операции идет без участия кеша. Учитывая, что файл, который мы считываем, очень большой, то операция чтения и записи на диск будет очень не выгодной.

Когда мы читаем через vtpc - мы читаем сохраняем файлы в наш созданный кеш, где идет политика вытеснения LRU. И не смотря на то, что мы читаем случайные позиции, то всегда есть шанс, что программа попадает в ту же страницу, что уже в кеше. В таком случае программа не загружает из диска, а грузит уже из кеша. Что ускоряет время чтения и записи.

То есть главная гипотеза:

Программы с vtpc будут работать быстрее, чем с системными вызовами.

Результат

=== РЕЗУЛЬТАТЫ СРАВНЕНИЯ ===

VTPC Cache:

Время выполнения: 3367 тактов (0.003 сек)

Загрузка CPU: user=2.6%, system=0.8%, wait=0.1%

Переключения контекста: 14875

Процессов: 0

System Calls (0_DIRECT):

Время выполнения: 494021 тактов (0.494 сек)

Загрузка CPU: user=2.6%, system=0.8%, wait=0.1%

Переключения контекста: 72818

Процессов: 0

При всех запусках показатель времени работы с системными файлами меньше, чем времени работы с vtpc